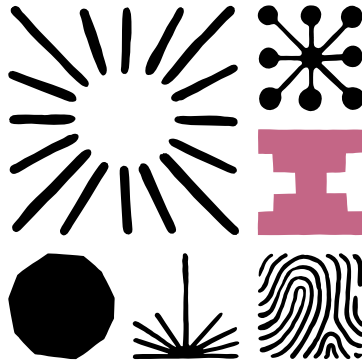


用并行运行的 Claude 团队构建 C 编译器

Building a C compiler with a team of parallel Claudes

Anthropic · 2026-02-05 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



我们让 Opus 4.6 以 Agent 团队的方式构建一个 C 编译器，随后便基本放手了。这个实验让我们对自主软件开发的未来有了以下认识。

作者为我们安全保障团队的研究员 *Nicholas Carlini*。

我一直在试验一种监督语言模型的新方法，我们称之为“Agent 团队”。

在 Agent 团队中，多个 Claude 实例在共享代码库上并行工作，无需人类主动介入。这种方法极大扩展了大语言模型 Agent 能够完成的工作范围。

为了对它进行压力测试，我让 16 个 Agent 从零开始用 Rust 编写一个能够编译 Linux 内核的 C 编译器。经过近 2,000 次 Claude Code 会话、花费约 20,000 美元 API 费用后，这个 Agent 团队产出了一个十万行代码的编译器，可以在 x86、ARM 和 RISC-V 上构建 Linux 6.9。

这个编译器本身就是一个有趣的成果，但本文重点讨论我在为长时间自主运行的 Agent 团队设计运行框架时学到的经验：如何编写测试，让 Agent 在无人监督时仍然保持方向；如何组织工作，让多个 Agent 可以并行推进；以及这种方法的能力上限在哪里。

让 Claude 长时间运行

Claude Code 等现有 Agent 辅助框架，需要操作者在线并随时参与协作。如果你要求它解决一个漫长而复杂的问题，模型可能先解决其中一部分，但最终会停下来，等待更多输入，例如问题、状态更新或澄清请求。

为了让它持续自主推进，我构建了一个运行框架，将 Claude 放进一个简单循环中（如果你见过 Ralph-loop，应该会很熟悉）。完成一项任务后，它会立刻开始下一项。（请在容器里运行，不要直接在你的实际机器上运行。）

```
#!/bin/bash

while true; do
  COMMIT=$(git rev-parse --short=6 HEAD)
  LOGFILE="agent_logs/agent_${COMMIT}.log"

  claude --dangerously-skip-permissions \
    -p "$(cat AGENT_PROMPT.md)" \
    --model claude-opus-X-Y &> "$LOGFILE"
done
```

在 Agent 提示词中，我会告诉 Claude 要解决什么问题，并要求它将问题拆成小块、记录正在处理的事项、确定下一步做什么，实际上就是一直做下去，直到完美为止。（关于最后一点，Claude 没有选择。这个循环会永远运行。不过，有一次我确实看到 Claude 意外执行了 `pkill -9 bash`，结果杀掉了自己，也终止了循环。哎呀！）

并行运行 Claude

并行运行多个实例，可以解决单 Agent 运行框架的两个弱点：

- 一个 Claude Code 会话一次只能做一件事。尤其当项目范围扩大时，同时调试多个问题的效率要高得多。
- 运行多个 Claude Agent 可以实现专业化分工。当几个 Agent 被分配去解决眼前的实际问题时，还可以调用其他专门的 Agent，例如维护文档、关注代码质量，或处理特定子任务。

我的 Claude 并行实现非常简陋。先创建一个新的裸 git 仓库，再为每个 Agent 启动一个 Docker 容器，将仓库挂载到 `/upstream`。每个 Agent 将本地副本克隆到 `/workspace`，完成后，再从各自的本地容器推送到上游。

为防止两个 Agent 同时尝试解决同一个问题，运行框架使用了一个简单的同步算法：

1. Claude 通过向 `current_tasks/` 写入一个文本文件来“锁定”任务，例如，一个 Agent 可以锁定 `current_tasks/parse_if_statement.txt`，另一个则锁定 `current_tasks/codegen_function_definition.txt`。如果两个 Agent 试图认领同一任务，git 的同步机制会迫使第二个 Agent 选择另一项。
2. Claude 处理任务，然后从上游拉取，合并其他 Agent 的变更，推送自己的变更，并移除锁。合并冲突经常发生，但 Claude 足够聪明，能够解决。
3. 无限运行的 Agent 生成循环在一个新容器中启动新的 Claude Code 会话，然后重复这一过程。

这是一个非常早期的研究原型。我还没有实现任何其他 Agent 间通信方式，也没有强制规定高层目标的管理流程。我没有使用编排 Agent。

相反，我让每个 Claude Agent 自行决定如何行动。在大多数情况下，Claude 会接手“下一个最明显”的问题。遇到难以解决的缺陷时，Claude 往往会持续维护一份文档，记录失败的方法和剩余任务。在项目的 git 仓库中，你可以翻阅历史，看到它如何为不同任务加锁。

使用 Claude Agent 团队编程的经验

辅助框架让 Claude 循环运行，但只有当 Claude 知道如何取得进展时，这个循环才有用。我的大部分精力都花在设计 Claude 周围的条件上，包括测试、环境和反馈，让它在没有我的情况下也能找到方向。下面是我发现的、在编排多个 Claude 实例时最有帮助的方法。

编写极高质量的测试

无论我给出什么问题，Claude 都会自主尝试解决。因此，任务验证器必须近乎完美，否则 Claude 就会解决错误的问题。改进测试框架，需要寻找高质量的编译器测试套件，为开源软件包编写验证器和构建脚本，观察 Claude 犯下的错误，并在识别出这些失败模式后设计新测试。

例如，在项目接近尾声时，Claude 开始在每次实现新功能时频繁破坏已有功能。为解决这一问题，我搭建了持续集成流水线，并实施更严格的约束，让 Claude 能更好地测试自己的工作，确保新提交不能破坏已有代码。

站在 Claude 的角度思考

我不得不不断提醒自己：这个测试框架是写给 Claude 的，而不是写给我自己的。这意味着，要重新思考我对测试应该如何传达结果的许多预设。

例如，每个 Agent 都会进入一个全新的容器，没有上下文，需要花不少时间了解状况，尤其是在大型项目中。甚至在开始测试之前，为了帮助 Claude 自助，我就加入了指令，要求它维护内容详尽的 README 和进度文件，并频繁更新当前状态。

我也始终记得，语言模型有一些固有限制，在这个项目中，设计需要针对这些限制进行调整，包括：

- **上下文窗口污染**：测试框架不应该打印成千上万字节的无用内容。它最多只应打印几行输出，并将所有重要信息写入文件，让 Claude 在需要时查找。日志文件应易于自动处理：如果有错误，Claude 应写下 ERROR，并把原因放在同一行，以便 grep 能够找到。提前计算汇总统计信息也很有帮助，这样 Claude 就不必重复计算。
- **时间感缺失**：Claude 无法感知时间，如果任其自行运行，它会乐此不疲地花上数小时跑测试，而不是推进工作。运行框架会低频打印增量进度（避免污染上下文），并提供默认的 `--fast` 选项，只运行随机抽取的 1% 或 10% 测试。这个子样本对每个 Agent 是固定的，但在不同虚拟机之间随机变化，因此 Claude 整体仍能覆盖所有文件，而每个 Agent 也都能准确识别回归问题。

让并行变得容易

当有许多不同的测试失败时，并行化很简单：每个 Agent 选择一个不同的失败测试来处理。测试套件通过率达到了 99% 后，各 Agent 开始分别让不同的小型开源项目（如 SQLite、Redis、libjpeg、MQuickJS、Lua）成功编译。

但当 Agent 开始编译 Linux 内核时，它们卡住了。与包含数百个独立测试的测试套件不同，编译 Linux 内核是一个庞大的单一任务。每个 Agent 都会碰到同一个缺陷、修复同一个缺陷，然后互相覆盖对方的改动。运行 16 个 Agent 并没有帮助，因为它们都困在同一个任务上。

解决办法是使用 GCC 作为运行时可调用、已知正确的编译器参考实现，进行对照。我编写了一个新的测试框架，随机选择内核的大部分文件交给 GCC 编译，只将其余文件交给 Claude 的 C 编译器。如果内核运行正常，那么问题就不在 Claude 编译的这部分文件中。如果出错，就再用 GCC 重新编译其中一部分文件，进一步缩小范围。这样，每个 Agent 都能并行工作，在不同文件中修复不同缺陷，直到 Claude 的编译器最终能够编译所有文件。（这种方法奏效后，仍然需要采用 delta debugging 技术，找出那些单独处理时正常、组合在一起却失败的文件对。）

多种 Agent 角色

并行也带来了专业化分工。大语言模型编写的代码经常重复实现已有功能，因此我让一个 Agent 负责合并它发现的重复代码。另一个负责提高编译器本身的性能，第三个负责提高生成的编译后代码的执行效率。我还让一个 Agent 从 Rust 开发者的角度审视项目设计，并进行结构调整，以改善整体代码质量；另一个则负责文档。

对 Agent 团队的能力极限进行压力测试

这个项目是作为能力基准设计的。我希望对大语言模型今天勉强能够实现的能力上限进行压力测试，帮助我们为未来模型能够可靠做到的事情作好准备。

我一直将 C 编译器项目用作整个 Claude 4 模型系列的基准。和此前的项目一样，我先起草需求：一个从零构建、没有依赖、兼容 GCC、能够编译 Linux 内核，并且设计上支持多个后端的优化编译器。虽然我指定了某些设计方面的要求，例如应使用 SSA 中间表示（IR），以支持多轮优化，但没有详细说明该如何实现。

此前的 Opus 4 模型仅勉强能够产出可用的编译器。Opus 4.5 第一次跨过了一个门槛，能够构建一个通过大型测试套件的可用编译器，但它仍然无法编译任何真正的大型项目。对 Opus 4.6，我的目标是再次测试极限。

评估

在两周内近 2,000 次 Claude Code 会话中，Opus 4.6 消耗了 20 亿个输入 token，生成了 1.4 亿个输出 token，总成本略低于 20,000 美元。即使与最昂贵的 Claude Max 套餐相比，这也是一个极其昂贵的项目。但这个总额，只占我亲自完成它所需成本的一小部分，更不用说投入一整个团队了。

这是一个净室实现，Claude 在整个开发过程中始终没有互联网访问权限；它只依赖 Rust 标准库。这个十多万行代码的编译器，可以在 x86、ARM 和 RISC-V 上构建能够启动的 Linux 6.9。它还能编译 QEMU、FFmpeg、SQLite、postgres 和 redis，并且在包括 GCC torture 测试套件在内的大多数编译器测试套件上，达到 99% 的通过率。它也通过了开发者的终极检验：可以编译并运行 Doom。

不过，这个编译器并非没有局限，包括：

- 它缺少 Linux 启动并离开实模式所需的 16 位 x86 编译器，因此这一步会调用 GCC；x86_32 和 x86_64 编译器则是它自己的。

- 它没有自己的汇编器和链接器；这是 Claude 最后才开始自动化实现的部分，目前仍有一些缺陷。演示视频使用的是 GCC 汇编器和链接器。
- 编译器能成功构建许多项目，但并非所有项目。它还不能直接替代成熟的编译器。
- 生成的代码效率不高。即使开启所有优化，它生成的代码仍然不如 GCC 关闭所有优化时生成的代码高效。
- Rust 代码质量尚可，但远远达不到 Rust 专家的水平。

最终得到的编译器已经接近 Opus 的能力极限。我曾非常努力地尝试解决上述几项局限，但未能完全成功。新功能和缺陷修复经常会破坏已有功能。

一个特别困难的例子是：Opus 无法实现启动进入 16 位实模式所需的 16 位 x86 代码生成器。虽然编译器可以通过 66/67 操作码前缀输出正确的 16 位 x86 代码，但编译后的结果超过 60kb，远高于 Linux 强制规定的 32k 代码上限。因此，Claude 在这里干脆“作弊”，在这个阶段调用 GCC。（这只发生在 x86 上。对于 ARM 或 RISC-V，Claude 的编译器完全可以独立完成编译。）

编译器源码已经公开。可以下载、阅读代码，并在你喜欢的 C 项目上试用。我一直发现，理解语言模型能做什么的最佳方式，就是将它们推到极限，再研究它们从哪里开始失效。接下来几天，我会继续让 Claude 推送新改动；如果你愿意，可以持续关注它解决这些局限的尝试。

展望未来

每一代语言模型，都会带来与之协作的新方式。早期模型适合在 IDE 中用 Tab 进行补全。不久后，模型就能根据文档字符串补全整个函数体。Claude Code 的发布让 Agent 走向主流，使开发者能够与 Claude 结对编程。但所有这些产品都基于同一个假设：用户定义任务，大语言模型运行几秒或几分钟并返回答案，然后用户提供后续输入。

Agent 团队展示了自主实现完整复杂项目的可能性。这让我们这些工具使用者，可以为自己设定更有雄心的目标。

我们仍处于早期阶段，而完全自主开发带来了真实风险。当人类在开发过程中与 Claude 一起工作时，可以确保质量始终如一，并实时发现错误。对于自主系统，人们很容易看到测试通过，就认为任务完成了，但事实很少如此。我以前做过渗透测试，利用大型公司产品中的漏洞，因此想到程序员可能部署从未亲自验证过的软件，我确实感到担忧。

所以，这个实验让我兴奋，也让我不安。构建这个编译器，是我最近做过的最有趣的事情之一，但我没想到在 2026 年这么早的时候，就已经如此接近可行。语言模型，以及我们与模型交互所用辅助框架的快速进步，为编写海量新代码打开了大门。我预计正面应用会多于负面应用，但我们正在进入一个新世界，需要用新的策略来安全前行。

致谢

特别感谢 Josef Bacik、Edwin Chen、Bernardo Meurer Costa、Jake Eaton、Dan Kelley、Felix Klock、Jannet Park、Steve Weis，以及 Anthropic 的许多其他同事提供的帮助与贡献。